



## **Title: Lab Project**

Course Code : CSE282.4

Course Title : **Programming Language II Lab**

Department : CSE

Faculty Name : **Mr.Md.Ayon Mia**

Work Place : **SOUTH EAST UNIVERSITY**

Assignment Submission Date : **03 June 2026**

### Submitted By

 **Group Tehreek e Muhandis**

| SL | Student Name         | ID            |
|----|----------------------|---------------|
| 1  | SAIFUL SIDDIKY LABIB | 2024100000469 |
| 2  | ALID HOSSAIN SHUVO   | 2024100000072 |
| 3  | MD. RAYHAN KAMAL     | 2024100000250 |
| 4  | ARMAN HOSSAIN SAMI   | 2024100000448 |
| 5  | TARAQ AZIZ           | 2024100000244 |

# **Lost & Found Management System (LFMS).**

## **1. INTRODUCTION:**

### **Project Overview:**

The Lost & Found Management System (LFMS) is a Java-based application designed to assist users in reporting lost items, registering found items, and submitting ownership claims. The system provides separate functionalities for regular users and administrators to ensure secure and organized management of lost-and-found records.

### **Project Objectives:**

1. Provide a platform for reporting lost items.
2. Allow users to register found items.
3. Enable ownership claim submission and verification.
4. Maintain secure user authentication and authorization.
5. Apply Object-Oriented Programming (OOP) principles in software development.

## 2. SOFTWARE DEVELOPMENT PROCESS:

The project requirements were analyzed and divided into the following modules:

### User Module:

1. Registration
2. Login
3. Profile management

### Item Management Module:

1. Add lost item
2. Add found item
3. View items

### Claim Management Module:

1. Submit claim
2. Review claim status

### Admin Module:

1. Approve claims
2. Reject claims
3. Manage records

### 3. DEVELOPMENT METHODOLOGY:

An incremental development approach was followed.

#### Phase 1

- Model Classes

#### Phase 2

- Service Classes

#### Phase 3

- Authentication System

#### Phase 4

- Claim Management

#### Phase 5

- User Interface Integration(GUI)

#### Phase 6

- Testing and Bug Fixing

### 4. SYSTEM DESIGN:

#### Package Structure:

```
src
├── model
├── service
├── exception
├── gui
└── main
```

### **1.Model Package**

Stores system entities.

### **2.Service Package**

Contains business logic.

### **3.Exception Package**

Handles custom exceptions.

### **4.GUI Package**

Contains graphical user interface components.

### **5.Main Package**

Application entry point.

## **5.SYSTEM ARCHITECTURE:**

The application follows a layered architecture:

User Interface Layer



Service Layer



Model Layer

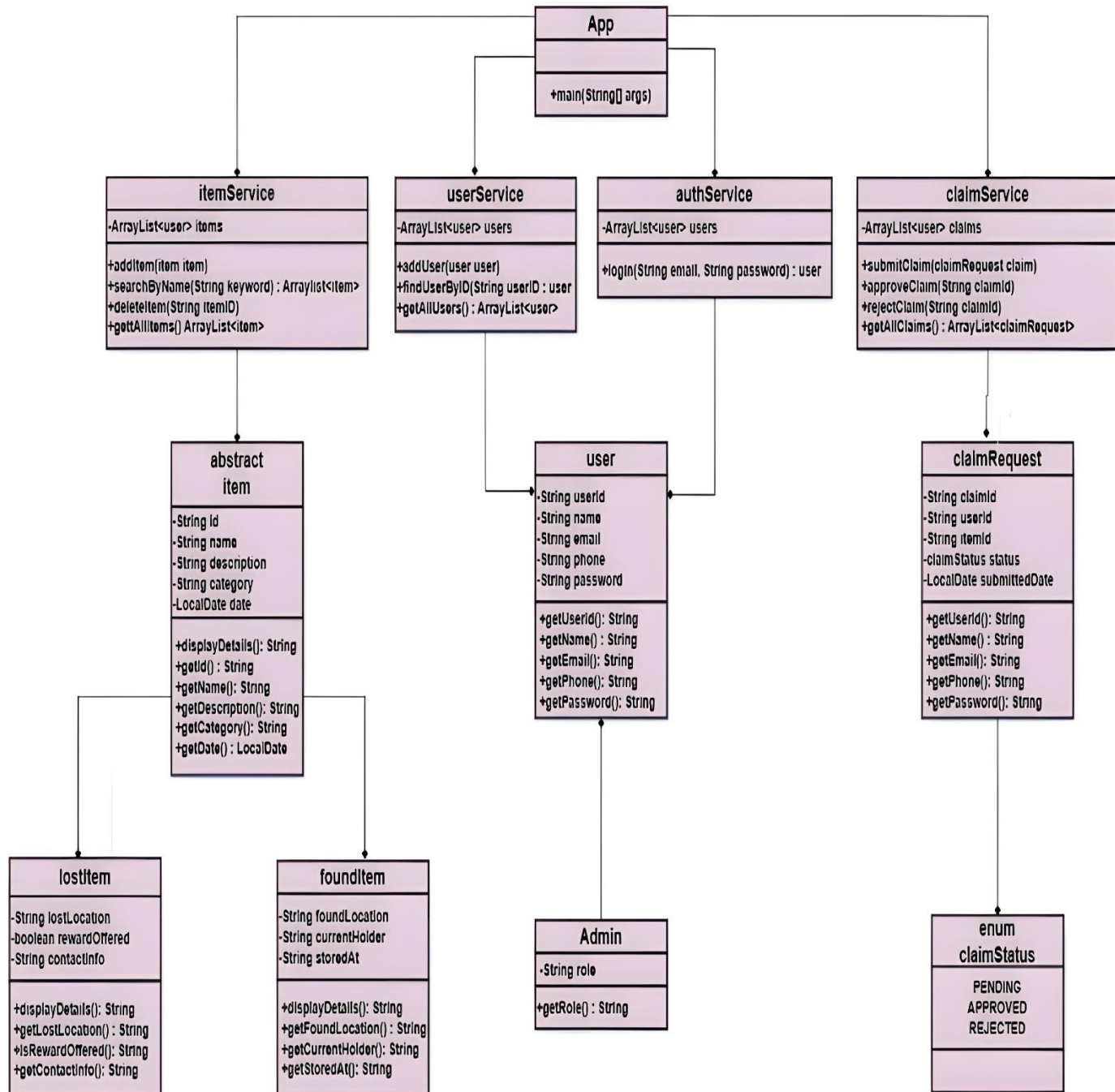


File Storage Layer

## 6.CLASS DESIGN

| CLASS              | DESCRIPTION                                                                      |
|--------------------|----------------------------------------------------------------------------------|
| App                | The main driver function that run all the other classes and methods .            |
| Item<br>(abstract) | Base class defining shared item fields.                                          |
| LostItem           | Extends Item with lostLocation, rewardOffered, contactInfo.                      |
| FoundItem          | Extends Item with foundLocation, storedAt, currentHolder.                        |
| User               | Holds personal details and item-related actions.                                 |
| Admin              | Inherits from User with additional privileges (approve/reject claims).           |
| ClaimStatus        | To verify if the item has been claimed, Approved or Pending                      |
| ClaimRequest       | Links users and items through claim processing and validation.                   |
| authService        | To make sure that users don't accidentally get admin access .                    |
| ClaimService       | Making sure that the claim process is well documented and valid via exception .  |
| itemService        | Ensuring that the item input is valid and no duplicates are made via exception . |
| userService        | Ensuring that the user input is valid and no duplicates are made via exception . |

## 7. CLASS DIAGRAM:



## **8. OOP Concept Mapping:**

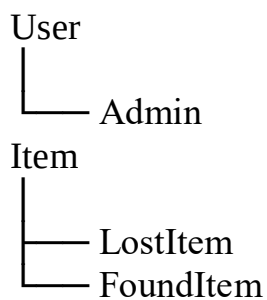
### **Encapsulation:**

Encapsulation was achieved by declaring class attributes as private and providing controlled access through getter and setter methods.

Classes demonstrating encapsulation:

- ❖ User
- ❖ Admin
- ❖ Item
- ❖ LostItem
- ❖ FoundItem
- ❖ ClaimRequest

### **Inheritance:**



## Abstraction:

The Item class was designed as an abstract class that defines common item properties while allowing subclasses to implement specific behavior.

Abstract Class:

- ❖ Item

Concrete Implementations:

- ❖ LostItem
- ❖ FoundItem

## Polymorphism:

Polymorphism was implemented through method overriding.

Examples:

- LostItem.displayDetails()
- FoundItem.displayDetails()

Objects of different subclasses can be treated as Item objects while executing their own specialized implementations.

## Exception Handling:

Custom exception handling was integrated into service classes to ensure data integrity and system reliability.

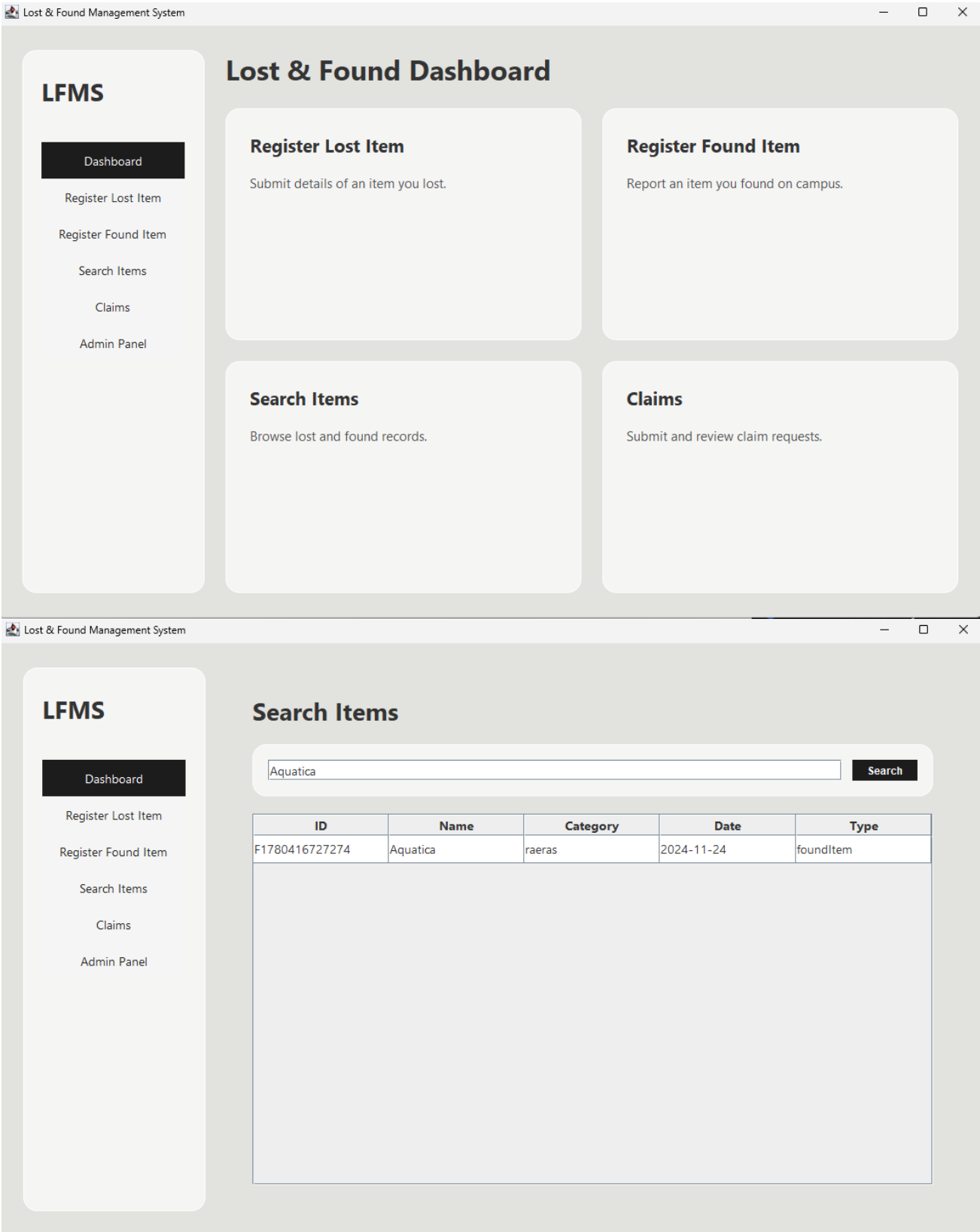
Service Classes Using Exceptions:

- UserService
- ItemService
- ClaimService
- AuthService

Examples:

- DuplicateUserException
- DuplicateItemException
- UserNotFoundException
- ItemNotFoundException
- ClaimNotFoundException

9.GUI SCREENSHOTS:



Lost & Found Management System

LFMS

Dashboard

Register Lost Item

Register Found Item

Search Items

Claims

Admin Panel

Register Lost Item

Submit details of an item you lost.

Item Name

Description

Category

Date Lost (YYYY-MM-DD)

Lost Location

Contact Information

☐ Reward Offered

Lost & Found Management System

LFMS

Dashboard

Register Lost Item

Register Found Item

Search Items

Claims

Admin Panel

Register Found Item

Submit details of a found item.

Item Name

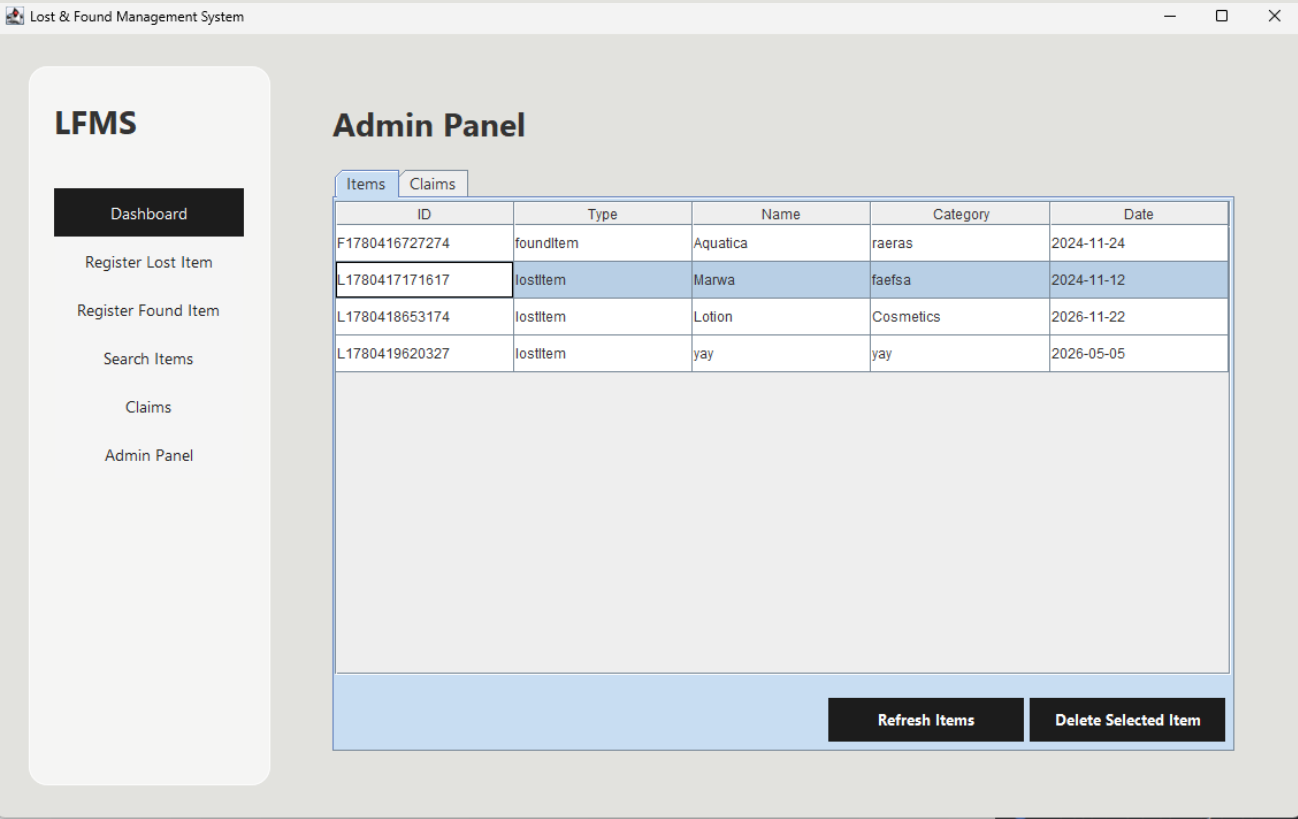
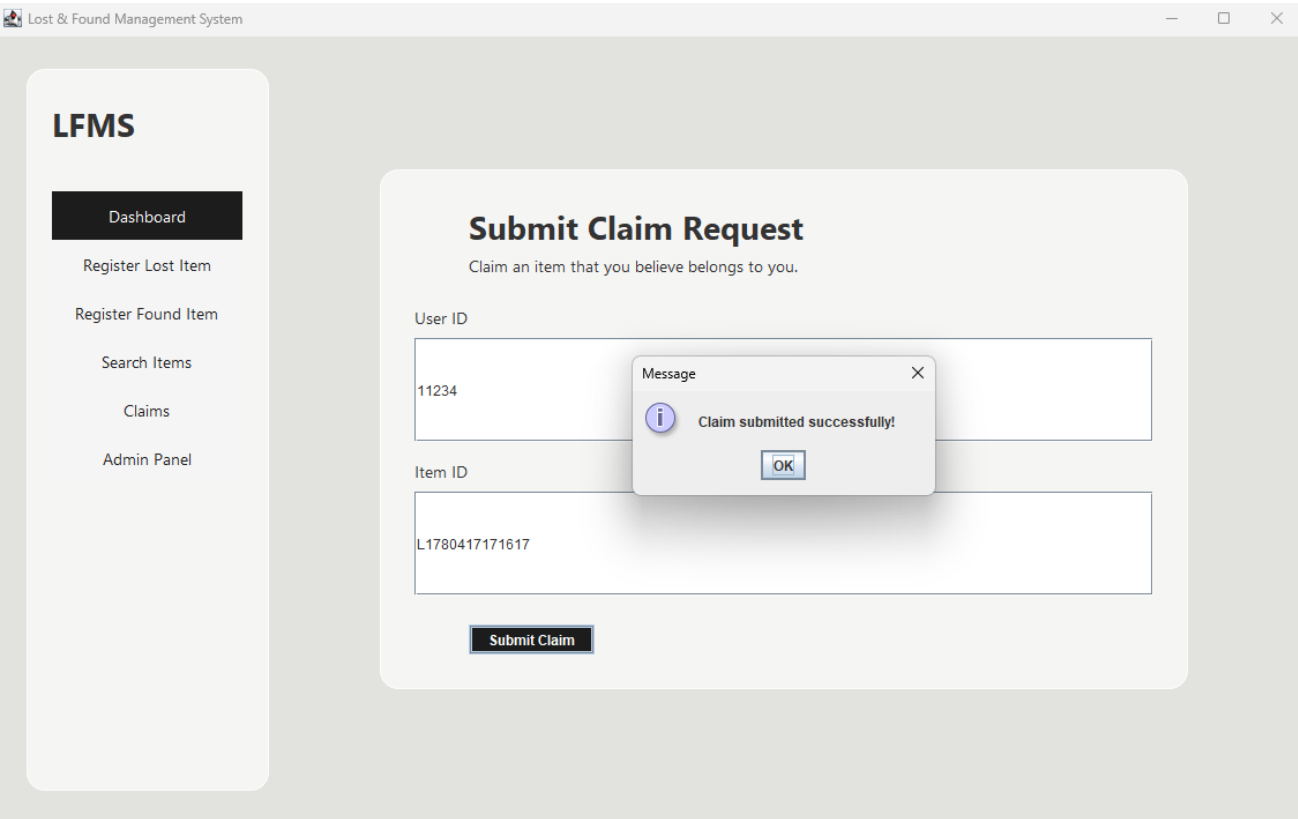
Description

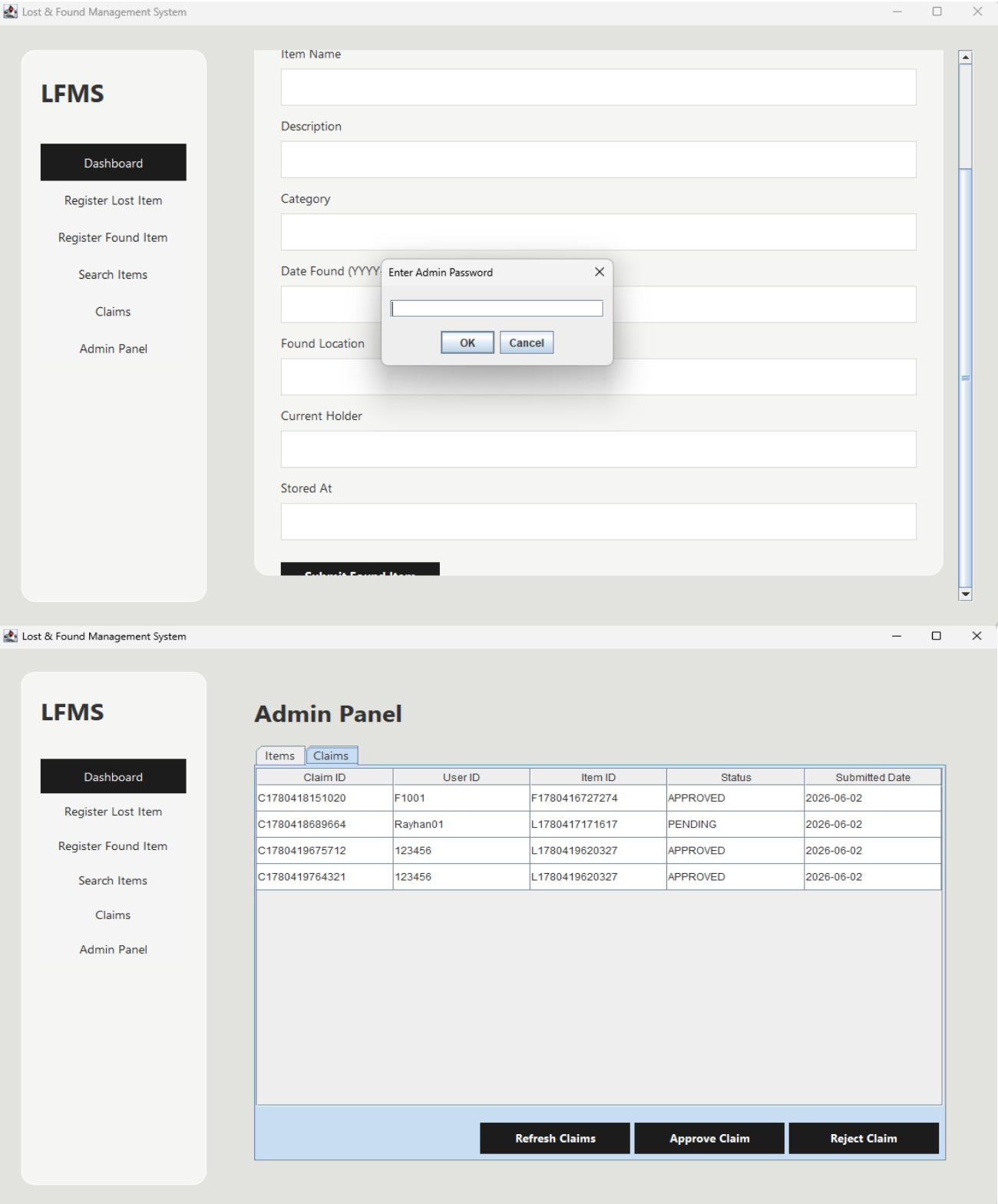
Category

Date Found (YYYY-MM-DD)

Found Location

Current Holder





## **9. REFLECTION ON CHALLENGES FACED:**

This project provided practical experience in applying Object-Oriented Programming concepts to a real-world problem. Through collaborative development, the team gained hands-on experience with Git version control, branch management, and software integration.

The project reinforced the importance of proper system design, modular programming, and teamwork. Challenges such as merge conflicts, access control implementation, exception handling, and GUI integration improved our problem-solving abilities and understanding of software engineering practices. The GUI development caused several issues during implementation, especially in terms of layout management and event handling. In addition, designing and implementing the admin panel was a particularly complex task due to the need for strict access control and multiple administrative functions.

Overall, the Lost & Found Management System successfully demonstrated the use of Java, OOP principles, file handling, exception management, GUI development, and collaborative software development techniques.

## **10. Conclusion:**

The Lost & Found Management System was successfully designed and implemented using Java and Object-Oriented Programming principles. The system fulfills its primary objectives of managing lost items, found items, user authentication, and ownership claims. The project also provided valuable experience in software design, implementation, testing, and team-based development using Git and GitHub.

GITHUB REPOSITORY LINK :

**<https://github.com/LABIBSIDDIKY/CSE281.4-JAVA-PROJECT>**